LAPORAN PRAKTIKUM MODUL 5

Analisis Kompleksitas Algoritma
Big O Notation, Time & Space Complexity

DISUSUN OLEH :

SAQYA RAHMATILLAH
2025573010131

Kelas : TI 1C
Mata Kuliah : Praktikum Algoritma dan Struktur Data
Jurusan : Teknologi Informasi dan Komputer
Prodi : Teknik Informatika
Dosen : Muhammad Reza Zulman, S.ST.,M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

2026

DAFTAR ISI

KATA PENGANTAR iv
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan Praktikum 1
1.4 Manfaat Praktikum 1
BAB II 2
DASAR TEORI 2
2.1 Pengertian Algoritma 2
2.2 Kompleksitas Algoritma 2
2.3 Time Complexity (Kompleksitas Waktu) 2
2.4 Big O Notation 2
2.5 Space Complexity (Kompleksitas Ruang) 3
2.6 Refactoring Kode 3
2.7 Hubungan Kompleksitas dengan Efisiensi Program 3
BAB III 4
LANGKAH-LANGKAH PRAKTIKUM 4
3.1 Persiapan Project 4
3.2 Langkah Pengerjaan 4
3.2.1 Kompleksitas Algoritma 4
3.2.2 Notasi Big O 6
Latihan 1: Identifikasi Kompleksitas 8
3.2.3 Space Complexity 9
Latihan 2: Space & Time Complexity 10
3.2.4 Mengenali Pola Kompleksitas 12
Tugas 1: Analisis dan Refactor 14
Tugas 2: Visualisasi Pertembuhan Big O 16
3.3 Langkah-langkah Push ke GitHub 18
BAB IV 19
HASIL DAN PEMBAHASAN 19
4.1 Hasil Praktikum 19
4.2 Pembahasan 19
4.2.1 Intro Kompleksitas 19
4.2.2 Big O Notation 19

4.2.3 Space Complexity 19
4.2.4 Refactoring 19
BAB V 20
PENUTUP 20
5.1 Kesimpulan 20
5.2 Saran 20

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan Praktikum Algoritma dan Struktur Data ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk tugas praktikum yang bertujuan untuk memahami konsep dasar kompleksitas algoritma, meliputi *time complexity*, *space complexity*, serta penerapan *Big O Notation* dan *refactoring* dalam pemrograman menggunakan JavaScript. Dengan adanya praktikum ini, diharapkan penulis dapat lebih memahami bagaimana membuat program yang efisien dan optimal.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada semua pihak yang telah membantu dalam penyusunan laporan ini.

Saqya Rahmatillah

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, membuat program yang bisa berjalan saja sebenarnya belum cukup. Kita juga perlu memperhatikan apakah program tersebut efisien atau tidak. Efisiensi ini penting karena akan berpengaruh pada kecepatan program dan penggunaan memori, terutama ketika data yang diolah jumlahnya besar.

Misalnya, ada dua program yang menghasilkan output yang sama, tetapi yang satu lebih cepat dan lebih hemat memori. Maka program tersebut tentu lebih baik untuk digunakan. Untuk mengetahui hal ini, digunakan konsep analisis kompleksitas algoritma.

Pada praktikum ini, akan dipelajari tentang Big O Notation, serta bagaimana cara mengukur waktu eksekusi (Time Complexity) dan penggunaan memori (Space Complexity). Materi ini sangat penting karena menjadi dasar sebelum mempelajari struktur data yang lebih lanjut.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, maka rumusan masalah dalam praktikum ini adalah:

1. Apa yang dimaksud dengan kompleksitas algoritma?
2. Apa itu Time Complexity dan Space Complexity?
3. Apa yang dimaksud dengan Big O Notation?
4. Bagaimana cara menganalisis kompleksitas suatu algoritma?
5. Bagaimana cara membandingkan dua algoritma dari segi efisiensi?

1.3 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami mengapa analisis efisiensi algoritma itu penting
2. Menjelaskan konsep Time Complexity dan Space Complexity
3. Membaca dan menulis notasi Big O: O(1), O(log n), O(n), O(n log n), O(n2)
4. Menganalisis kompleksitas loop tunggal dan nested loop
5. Membandingkan efisiensi dua algoritma secara empiris menggunakan Date.now()
6. Mengenali pola kode yang memiliki kompleksitas buruk dan cara memperbaikinya

1.4 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah:

1. Mahasiswa dapat menulis kode yang lebih efisien
2. Mahasiswa dapat memahami performa program yang dibuat
3. Mahasiswa bisa memilih algoritma yang lebih baik
4. Menghindari penggunaan kode yang terlalu berat

BAB II

DASAR TEORI

2.1 Pengertian Algoritma

Algoritma adalah serangkaian langkah-langkah logis dan sistematis yang digunakan untuk menyelesaikan suatu masalah. Dalam pemrograman, algoritma menjadi dasar dalam pembuatan program agar solusi yang dihasilkan dapat berjalan dengan benar dan efisien.

Algoritma yang baik memiliki beberapa karakteristik, yaitu:

- Memiliki langkah yang jelas dan terurut
- Memiliki input dan output
- Bersifat efektif dan efisien
- Dapat diselesaikan dalam waktu terbatas

2.2 Kompleksitas Algoritma

Kompleksitas algoritma adalah ukuran untuk menilai seberapa efisien suatu algoritma dalam menyelesaikan masalah. Kompleksitas biasanya dilihat dari dua aspek utama:

- Waktu eksekusi (time complexity)
- Penggunaan memori (space complexity)

Tujuan analisis kompleksitas adalah untuk mengetahui performa algoritma, terutama ketika data yang diproses dalam jumlah besar.

2.3 Time Complexity (Kompleksitas Waktu)

Time complexity adalah ukuran yang menunjukkan seberapa lama waktu yang dibutuhkan oleh suatu algoritma untuk berjalan, tergantung pada ukuran input (n).

Dalam praktikum 01-intro-kompleksitas.js, diperkenalkan konsep dasar bagaimana jumlah operasi dalam algoritma mempengaruhi waktu eksekusi.

Semakin banyak langkah yang dilakukan, maka semakin lama waktu yang dibutuhkan. Oleh karena itu, penting untuk membuat algoritma yang efisien agar tidak membebani sistem.

2.4 Big O Notation

Big O Notation adalah cara untuk menyatakan kompleksitas waktu suatu algoritma dalam bentuk yang sederhana. Notasi ini digunakan untuk menggambarkan pertumbuhan jumlah operasi terhadap ukuran input.

Pada praktikum 02-kelas-big-o.js, dijelaskan beberapa jenis kompleksitas Big O, antara lain:

- O(1) – Constant Time
  Waktu eksekusi tetap, tidak tergantung pada ukuran input.
- O(n) – Linear Time
  Waktu bertambah seiring dengan bertambahnya jumlah data.
- $O(n^2)$ – Quadratic Time
  Waktu meningkat secara kuadrat, biasanya terjadi pada nested loop.

- O(log n) – Logarithmic Time
  Waktu bertambah secara logaritmik, biasanya pada algoritma pencarian efisien seperti binary search.

Big O membantu dalam membandingkan algoritma mana yang lebih efisien.

2.5 Space Complexity (Kompleksitas Ruang)

Space complexity adalah ukuran yang menunjukkan seberapa besar memori yang digunakan oleh suatu algoritma selama proses eksekusi.

Pada praktikum 03-space-complexity.js, dijelaskan bahwa penggunaan memori dapat berasal dari:

- Variabel
- Struktur data (array, object)
- Rekursi (stack)

Contoh:

- Algoritma dengan O(1) → menggunakan memori tetap
- Algoritma dengan O(n) → memori bertambah sesuai jumlah data

Semakin kecil penggunaan memori, maka semakin efisien algoritma tersebut.

2.6 Refactoring Kode

Refactoring adalah proses memperbaiki struktur kode tanpa mengubah fungsi utama dari program. Tujuannya adalah untuk membuat kode menjadi:

- Lebih rapi
- Mudah dibaca
- Lebih efisien
- Mudah dikembangkan

Pada praktikum 04-refactor.js, refactoring dilakukan dengan cara:

- Mengurangi kode yang berulang
- Menyederhanakan logika program
- Menggunakan fungsi yang lebih efisien

Refactoring sangat penting dalam pengembangan perangkat lunak agar kode tetap terorganisir dan mudah dipahami.

2.7 Hubungan Kompleksitas dengan Efisiensi Program

Kompleksitas algoritma sangat berpengaruh terhadap performa program. Algoritma dengan kompleksitas rendah akan:

- Lebih cepat dijalankan
- Menggunakan memori lebih sedikit
- Lebih cocok untuk data besar

Sebaliknya, algoritma dengan kompleksitas tinggi dapat menyebabkan program menjadi lambat dan tidak efisien.

Oleh karena itu, pemilihan algoritma yang tepat sangat penting dalam proses pengembangan program.

BAB III

LANGKAH-LANGKAH PRAKTIKUM

3.1 Persiapan Project

1. Buka Git Bash. Kemudian navigasikan ke folder repository GitHub kamu yang sudah di-clone.

2. Buat folder baru bernama praktikum-5.

3. Inisialisasi npm project. Perintah ini akan membuat file package.json secara otomatis.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData (main)
$ mkdir praktikum-5

USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData (main)
$ cd praktikum-5

USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ npm init -y
Wrote to C:\Users\USER\2025573010131_PratikumAlgoritmaDanStrukturData\praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

3.2 Langkah Pengerjaan

3.2.1 Kompleksitas Algoritma

1. Buat file baru bernama 01-intro-kompleksitas.js.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js
```

2. Buka file 01-intro-kompleksitas.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 01-intro-kompleksitas.js
// Mengukur perbedaan performa algoritma secara empiris
// Fungsi bantu: ukur waktu eksekusi
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}
const N = 100_000; // ukuran data: 100 ribu
// --- Algoritma A: O(n) — loop satu kali ---
function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}
// --- Algoritma B: O(1) — rumus matematika ---
function jumlahkanRumus(n) {
  return (n * (n + 1)) / 2;
}
// --- Algoritma C: O(n2) — loop bersarang ---
function cariPasangan(arr) {
  const pasangan = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
    }
  }
  return pasangan;
}
const data = Array.from({ length: 5000 }, (_, i) => i);
console.log("=== Perbandingan Waktu Eksekusi ===");
ukurWaktu("O(1) jumlahkanRumus ", () => jumlahkanRumus(N));
ukurWaktu("O(n) jumlahkanLinear ", () => jumlahkanLinear(N));
ukurWaktu("O(n2) cariPasangan ", () => cariPasangan(data));
console.log("\nHasil sama?", jumlahkanLinear(100) === jumlahkanRumus(100));
```

3. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 1 ms
O(n2) cariPasangan : 20 ms

Hasil sama? true
```

3.2.2 Notasi Big O

1. Buat file baru bernama 02-kelas-big-o.js di dalam folder praktikum-5/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ touch 02-destructuring-spread.js
```

2. Buka file 02-kelas-big-o.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 02-kelas-big-o.js
// Contoh konkret setiap kelas kompleksitas Big O
// ── O(1) — Konstan ──────────────────────────────
function ambilPertama(arr) {
  return arr[0];
} // O(1)
function tambahkanItem(arr, item) {
  arr.push(item);
} // O(1)
function isGenap(n) {
  return n % 2 === 0;
} // O(1)
// ── O(log n) — Logaritmik ───────────────────────
function binarySearch(arr, target) {
  let kiri = 0,
    kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
  }
  return -1;
}
// ── O(n) — Linear ───────────────────────────────
function cariMax(arr) {
  let maks = arr[0];
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}
// ── O(n2) — Kuadratik ───────────────────────────
function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length - i - 1; j++)
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
  return a;
}
// ── O(2n) — Eksponensial ────────────────────────
function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2); // dua panggilan rekursif!
}
```

```javascript
// ── Demonstrasi ──────────────────────────────────────
console.log("=== O(1) — selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42)); // true
console.log("\n=== O(log n) — Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452); // cari di 1 juta data
console.log("\n=== O(n) — Linear Search ===");
console.log(
  "Max dari 1000 elemen:",
  cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
);
console.log("\n=== O(n2) — Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
console.log("\n=== O(2n) — Fibonacci Rekursif (n kecil!) ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
console.log("");
// Demonstrasi betapa lambatnya O(2n)
console.log("\nWaktu fib(35) O(2n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");
console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
t = Date.now();
fibMemo(35);
console.log(Date.now() - t, "ms");
```

3. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ node 02-kelas-big-o.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 999

=== O(n2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
99 ms
Waktu fib(35) O(n) memoization:
0 ms
```

Latihan 1: Identifikasi Kompleksitas

1. Buat file baru latihan-1.js.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ touch latihan-1.js
```

2. Ketikkan kode berikut.

```javascript
// Latihan 1: Identifikasi Kompleksitas

// Fungsi A
// Big O: O(1)
// Alasan: Operasi konstan, tidak bergantung pada n
function fnA(n) {
  return n * 2;
}

// Fungsi B
// Big O: O(n^2)
// Alasan: Dua perulangan bersarang (nested loop)
function fnB(n) {
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      // console.log(i, j);
    }
  }
}

// Fungsi C
// Big O: O(log n)
// Alasan: i dikali 2 setiap iterasi (pertumbuhan eksponensial)
function fnC(n) {
  for (let i = 1; i < n; i *= 2) {
    // console.log(i);
  }
}

// Fungsi D
// Big O: O(n^3)
// Alasan: Tiga perulangan bersarang
function fnD(n) {
  let arr = Array.from({ length: n }, (_, i) => i);

  arr.forEach((x) => {
    arr.forEach((y) => {
      arr.forEach((z) => {
        // console.log(x, y, z);
      });
    });
  });
}
```

```
// Fungsi untuk menghitung waktu eksekusi
function hitungKompleksitas(n, fn) {
  const start = Date.now();
  fn(n);
  const end = Date.now();
  console.log(fn.name + " => " + (end - start) + " ms");
}

// Pengujian
const n = 1000;

hitungKompleksitas(n, fnA);
hitungKompleksitas(n, fnB);
hitungKompleksitas(n, fnC);

// Hati-hati: fnD sangat berat jika n besar
hitungKompleksitas(50, fnD);
```

3. Simpan dan jalankan

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ node latihan-1.js
fnA => 0 ms
fnB => 2 ms
fnC => 1 ms
fnD => 1 ms
```

3.2.3 Space Complexity

1. Buat file baru bernama 03-space-complexity.js di dalam folder praktikum-5/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ touch 03-space-complexity.js
```

2. Buka file 03-space-complexity.js di VS Code, lalu ketikkan kode berikut.

```
// 03-space-complexity.js
// --- O(1) Space: hanya pakai variabel tambahan konstan ---
function jumlahArray(arr) {
  let total = 0; // hanya 1 variabel tambahan
  for (const x of arr) total += x;
  return total;
}
// --- O(n) Space: membuat struktur baru sebesar input ---
function duplikasiArray(arr) {
  const baru = []; // array baru tumbuh seiring arr
  for (const x of arr) baru.push(x * 2);
  return baru;
}
```

```
// --- O(n) Space: rekursi (call stack) ---
function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1); // n frame di call stack
}
// --- O(1) Space: faktorial iteratif ---
function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
  return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log("Jumlah array :", jumlahArray(arr)); // O(1) space
console.log("Duplikasi array:", duplikasiArray(arr)); // O(n) space
console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
// Visualisasi: hitung elemen unik (O(n) space)
function hitungUnik(arr) {
  const seen = new Set(); // Set tumbuh hingga n elemen
  for (const x of arr) seen.add(x);
  return seen.size;
}
const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log("Elemen unik:", hitungUnik(dataAcak)); // 7
```

3. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
```

Latihan 2: Space & Time Complexity

1. Tambahkan kode di bawah file 04-space-complexity.js.

```
// Latihan 2: Space & Time Complexity

// Fungsi 1: Cara Lambat (Nested Loop)
// Big O Time: O(n^2)
// Big O Space: O(1)
// Alasan: dua loop bersarang, tanpa struktur tambahan
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
```

```
  return null;
}

// Fungsi 2: Cara Cepat (Menggunakan Set)
// Big O Time: O(n)
// Big O Space: O(n)
// Alasan: satu loop, tapi pakai Set untuk menyimpan data
function cariPasanganCepat(arr, target) {
  const seen = new Set();

  for (let i = 0; i < arr.length; i++) {
    const selisih = target - arr[i];

    if (seen.has(selisih)) {
      return [selisih, arr[i]];
    }

    seen.add(arr[i]);
  }

  return null;
}

// Pengujian kecil (sesuai soal)
const smallArr = [2, 7, 11, 15];
const target = 9;

console.log("Hasil Lambat:", cariPasanganLambat(arr, target)); // [2,7]
console.log("Hasil Cepat :", cariPasanganCepat(arr, target)); // [2,7]

// Pengujian data besar (50.000)

// generate data acak
const arrBesar = Array.from({ length: 50000 }, () =>
  Math.floor(Math.random() * 100000),
);

const targetBesar = -1; // kemungkinan tidak ada

// fungsi ukur waktu
function ukurWaktu(fn, arr, target) {
  const start = Date.now();
  const hasil = fn(arr, target);
  const end = Date.now();
  console.log(fn.name + " => " + (end - start) + " ms | hasil:", hasil);
}

// Jalankan pengujian

console.log("\n=== Pengujian Data Besar ===");

// Fungsi lambat pakai data kecil agar tidak hang
ukurWaktu(cariPasanganLambat, arrBesar.slice(0, 2000), targetBesar);

// Fungsi cepat tetap pakai 50.000 data
ukurWaktu(cariPasanganCepat, arrBesar, targetBesar);

// Diskusi

// Fungsi terbaik: cariPasanganCepat
// Karena:
// - Time complexity lebih baik (O(n))
// - Lebih cepat untuk data besar

// Trade-off:
// - Lambat: hemat memori (O(1)) tapi sangat lama
// - Cepat: lebih boros memori (O(n)) tapi jauh lebih efisien
```

3. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7


 latihan
Hasil Lambat: [ 1, 8 ]
Hasil Cepat : [ 4, 5 ]

=== Pengujian Data Besar ===
cariPasanganLambat => 5 ms | hasil: null
cariPasanganCepat => 5 ms | hasil: null
```

3.2.4 Mengenali Pola Kompleksitas

1. Buat file baru bernama 04-refactor.js di dalam folder praktikum-5/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ touch 04-refactor.js
```

2. Buka file 04-refactor.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 04-refactor.js — Refactoring kode berperforma buruk
// ==============================================
// SKENARIO 1: Cek duplikat dalam array
// ==============================================
// BURUK: O(n2) — nested loop
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j]) return true;
  return false;
}
// BAIK: O(n) — gunakan Set
function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}
// ==============================================
// SKENARIO 2: Frekuensi kemunculan elemen
// ==============================================
```

```javascript
// BURUK: O(n2) — indexOf di dalam loop
function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x); // O(n) di dalam loop O(n)
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}
// BAIK: O(n) — gunakan object/Map sebagai counter
function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
  return counter;
}
// ── Benchmark ──────────────────────────────
const besar = Array.from({ length: 20000 }, () =>
  Math.floor(Math.random() * 1000),
);
let t = Date.now();
adaDuplikatLambat(besar);
console.log("Duplikat LAMBAT O(n2):", Date.now() - t, "ms");
t = Date.now();
adaDuplikatCepat(besar);
console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");
const kata = ["a", "b", "a", "c", "b", "a", "d"];
console.log("\nFrekuensi:", frekuensiCepat(kata));
```

3. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

Tugas 1: Analisis dan Refactor

1. Buat file tugas/tugas-1.js tugas/tugas-2.js.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ mkdir tugas

USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

2. Ketikkan kode berikut.

```javascr
// TUGAS 1: Analisis dan Refactor

// Fungsi A: Intersection (irisan array)
// Versi O(n^2)
// Time: O(n^2)
// Space: O(1)
function intersectionLambat(arr1, arr2) {
  const result = [];

  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j] && !result.includes(arr1[i])) {
        result.push(arr1[i]);
      }
    }
  }
  return result;
}

// Versi O(n) menggunakan Set
// Time: O(n)
// Space: O(n)
function intersectionCepat(arr1, arr2) {
  const set = new Set(arr2);
  const result = [];

  for (let item of arr1) {
    if (set.has(item) && !result.includes(item)) {
      result.push(item);
    }
  }
  return result;
}

// Fungsi B: Kelompok Anagram
// Time: O(n * k log k)
// Space: O(n)

function kelompokAnagram(arr) {
  const map = {};

  for (let kata of arr) {
    const key = kata.split("").sort().join("");

    if (!map[key]) {
      map[key] = [];
    }

    map[key].push(kata);
  }
  return Object.values(map);
}

```

```javascript
// Fungsi C: Two Sum (jumlah target)
// Versi O(n^2)
// Time: O(n^2)
// Space: O(1)
function twoSumLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return true;
      }
    }
  }
  return false;
}

// Versi O(n log n) (sort + two pointer)
// Time: O(n log n)
// Space: O(1)
function twoSumCepat(arr, target) {
  arr.sort((a, b) => a - b);

  let left = 0;
  let right = arr.length - 1;

  while (left < right) {
    const sum = arr[left] + arr[right];

    if (sum === target) return true;
    if (sum < target) left++;
    else right--;
  }
  return false;
}

// Pengujian
console.log("Intersection Lambat:", intersectionLambat([1, 2, 3], [2, 3, 4]));
console.log("Intersection Cepat:", intersectionCepat([1, 2, 3], [2, 3, 4]));

console.log(
  "Anagram:",
  kelompokAnagram(["eat", "tea", "tan", "ate", "nat", "bat"]),
);

console.log("Two Sum Lambat:", twoSumLambat([2, 7, 11, 15], 9));
console.log("Two Sum Cepat:", twoSumCepat([2, 7, 11, 15], 9));
```

3. Simpan dan jalankan: node tugas/tugas-1.js.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5/tugas (main)
$ node tugas-1.js
Intersection Lambat: [ 2, 3 ]
Intersection Cepat: [ 2, 3 ]
Anagram: [ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
Two Sum Lambat: true
Two Sum Cepat: true
```

Tugas 2: Visualisasi Pertembuhan Big O

1. Ketikkan kode berikut.

```javascript
// TUGAS 2: Visualisasi Big O

// O(1)
function fn_O1(n) {
  return n + 1;
}

// O(n)
function fn_On(n) {
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += i;
  }
  return sum;
}

// O(n log n)
function fn_Onlogn(n) {
  let count = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 1; j < n; j *= 2) {
      count++;
    }
  }
  return count;
}

// O(n^2)
function fn_On2(n) {
  let count = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      count++;
    }
  }
  return count;
}

// Fungsi benchmark
function benchmarkSemua(ukuranData) {
  for (let n of ukuranData) {
    console.log("\nUkuran:", n);

    ukur(fn_O1, n);
    ukur(fn_On, n);
    ukur(fn_Onlogn, n);
    ukur(fn_On2, n);
  }
}

```

```javascript
// Helper ukur waktu
function ukur(fn, n) {
  const start = Date.now();
  fn(n);
  const end = Date.now();
  console.log(fn.name + " => " + (end - start) + " ms");
}

// Jalankan benchmark
benchmarkSemua([100, 500, 1000, 5000, 10000]);

// Observasi (untuk laporan):
// - O(1) hampir tidak berubah
// - O(n) naik linear
// - O(n log n) naik lebih cepat dari O(n)
// - O(n^2) sangat cepat membesar (paling lambat)
// - Hasil sesuai teori Big O
```

2. Simpan dan jalankan: node tugas/tugas-2.js.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5/tugas (main)
$ node tugas-2.js

Ukuran: 100
fn_O1 => 0 ms
fn_On => 0 ms
fn_Onlogn => 0 ms
fn_On2 => 0 ms

Ukuran: 500
fn_O1 => 0 ms
fn_On => 0 ms
fn_Onlogn => 0 ms
fn_On2 => 1 ms

Ukuran: 1000
fn_O1 => 0 ms
fn_On => 0 ms
fn_Onlogn => 0 ms
fn_On2 => 2 ms

Ukuran: 5000
fn_O1 => 0 ms
fn_On => 0 ms
fn_Onlogn => 1 ms
fn_On2 => 26 ms

Ukuran: 10000
fn_O1 => 0 ms
fn_On => 1 ms
fn_Onlogn => 0 ms
fn_On2 => 101 ms
```

3.3 Langkah-langkah Push ke GitHub

1. Buka terminal Git Bash, kemudian masuk ke folder project dengan perintah:

2. Tambahkan seluruh file dengan perintah:

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ git add .
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by CRLF the next time Git touches it
```

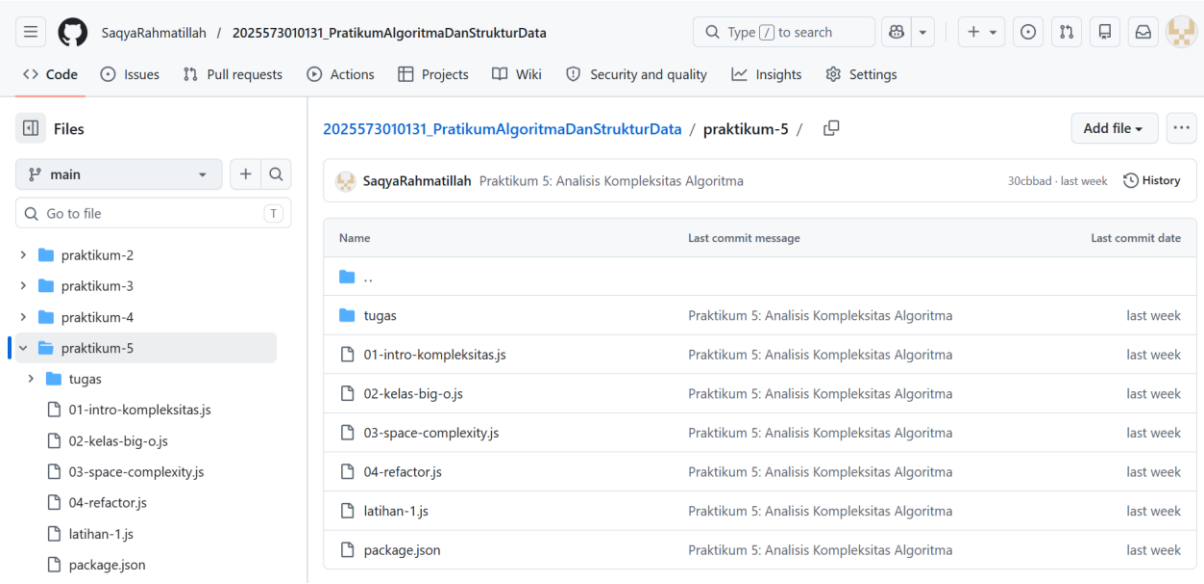
3. Lakukan commit dengan perintah:

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main 30cbbad] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 529 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-kelas-big-o.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js
```

4. Unggah (push) project ke GitHub dengan perintah:

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ git push
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 6.21 KiB | 706.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/SaqyaRahmatillah/2025573010131_PratikumAlgoritmaDanStrukturData.git
   186a777..30cbbad  main -> main
```

5. Project berhasil di unggah (push).



BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum

Pada praktikum ini, dilakukan beberapa percobaan menggunakan JavaScript untuk memahami konsep kompleksitas algoritma, baik dari segi waktu maupun ruang. File yang digunakan antara lain:

- 01-intro-kompleksitas.js
- 02-kelas-big-o.js
- 03-space-complexity.js
- 04-refactor.js

Hasil dari praktikum menunjukkan bahwa setiap algoritma memiliki tingkat kompleksitas yang berbeda tergantung pada jumlah operasi dan penggunaan memori.

4.2 Pembahasan

4.2.1 Intro Kompleksitas

Pada percobaan ini digunakan perulangan untuk melihat pengaruh jumlah data terhadap waktu eksekusi.
Hasilnya:

- Satu loop → $O(n)$
- Nested loop → $O(n^2)$

Artinya, semakin banyak perulangan, semakin lama waktu proses.

4.2.2 Big O Notation

Pengujian beberapa jenis kompleksitas menghasilkan:

- $O(1)$ → waktu tetap
- $O(n)$ → bertambah sesuai data
- $O(n^2)$ → meningkat lebih cepat

Kesimpulan: algoritma dengan kompleksitas kecil lebih efisien.

4.2.3 Space Complexity

Analisis menunjukkan:

- Variabel sederhana → $O(1)$
- Penggunaan array → $O(n)$

Semakin banyak data disimpan, semakin besar memori yang digunakan.

4.2.4 Refactoring

Refactoring membuat kode:

- Lebih singkat
- Lebih rapi
- Lebih efisien

Tidak mengubah fungsi utama, tetapi meningkatkan kualitas kode.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Algoritma memiliki tingkat kompleksitas yang berbeda-beda tergantung pada jumlah operasi yang dilakukan.
2. Big O Notation digunakan untuk mengukur efisiensi algoritma, baik dalam hal waktu maupun ruang.
3. Kompleksitas waktu (time complexity) berpengaruh terhadap kecepatan eksekusi program.
4. Kompleksitas ruang (space complexity) berpengaruh terhadap penggunaan memori.
5. Refactoring sangat penting untuk memperbaiki struktur kode tanpa mengubah fungsi utama program.

Dengan memahami konsep ini, mahasiswa dapat membuat program yang lebih efisien dan optimal.

5.2 Saran

Adapun saran yang dapat diberikan setelah melakukan praktikum ini adalah:

1. Mahasiswa perlu lebih banyak berlatih dalam menganalisis kompleksitas algoritma.
2. Dalam membuat program, sebaiknya memperhatikan efisiensi waktu dan memori.
3. Biasakan melakukan refactoring agar kode lebih rapi dan mudah dipahami.
4. Gunakan algoritma yang paling sesuai dengan kebutuhan agar performa program tetap optimal.